Lesson 2.3: Retrieval-augmented generation (RAG) and document indexing

What you learn

- **The pipeline.** Split documents into chunks, embed and index them, retrieve the best chunks for a question, give them to the model with the question, and ask it to answer only from them and cite the source.
- **Chunking.** Chunks of a few hundred words with a little overlap keep ideas together.
- **Failure modes.** Wrong chunk retrieved, answer not in the documents, or the model ignoring the context. Test with real questions and add 'say you do not know' to the instruction.
- **Privacy.** Index only documents you may use and respect who can see what.

Tools and resources (official sites, not checked live)

- Anthropic RAG guidance
- LlamaIndex

ASSIGNMENT

Practice for lesson 2.3

Business problem: Test a mini RAG.

Steps

1. Run the tested code.
2. Ask a question that is not in the documents.
3. Confirm it says it was not found.

Expected output: Answers cite a source, and unknown questions are refused.

SOLUTION (ANSWER KEY)

PYTHON

```
import re
from collections import Counter

STOP = {"the", "and", "for", "are", "you", "your", "how", "what", "does", "long", "take",
        "have", "with", "from", "that", "this", "after", "within", "each", "shown", "before",
"run", "our", "can", "will"}
DOCS = {
    "refund": "Refunds are processed within 7 working days after a written request.",
    "fees": "Course fees are shown on each program page and confirmed in writing before
payment.",
    "timings": "Live classes run on Saturdays and Sundays in the evening.",
}

def tokens(text):
    return [w for w in re.findall(r"[a-z]+", text.lower()) if len(w) > 2 and w not in STOP]

def retrieve(question, docs=DOCS, k=1):
    q = Counter(tokens(question))
    scored = []
    for key, text in docs.items():
```

```
        d = Counter(tokens(text))
        score = sum(min(q[w], d[w]) for w in q)
        scored.append((score, key, text))
    scored.sort(reverse=True)
    return [(key, text) for score, key, text in scored[:k] if score > 0]

def answer(question):
    hits = retrieve(question)
    if not hits:
        return "Not found in our documents. A team member will reply."
    key, text = hits[0]
    return f"{text} [source: {key}]"
```

The answer includes a source tag. The pizza question returns 'Not found', which is the safe behaviour.

Check yourself

1. Why cite the source?
   Answer: So users can check the answer.
2. What should the bot do when the answer is not in the documents?
   Answer: Say so and hand over to a person.

Lesson 2.3 of the **AI and Automation** lessons. See this lesson and its official links on skillnavora.in · All chapters and lessons · Open the full lesson guide. Tools, prices and plans change; confirm on each official site.